Project Design Phase-I

Solution Architecture

Date	10 March 2026
Project Name	UCab – Cab Booking Web Application

Solution Architecture

Solution architecture describes the structure of the UCab cab booking system and explains how different components of the system interact with each other. It connects the business problem (booking transportation easily) with the technology solution used to build the system.

The main goals of the UCab solution architecture are:

- Find the best technology solution for an online cab booking system.
- Describe how the frontend, backend, and database communicate with each other.
- Define system features such as user registration, cab booking, and admin management.
- Provide a clear structure that developers can follow during development and deployment.

System Components

Component	Description	Technology Used
Frontend Layer	The user interface where users interact with the UCab application. It displays pages such as home page, login, booking form, and dashboard.	React.js, Bootstrap
Backend Layer	The backend handles API requests, booking logic, authentication, and communication with the database.	Node.js, Express.js
Database Layer	The database stores all system data including users, cabs, drivers, bookings, and payment information.	MongoDB
Authentication System	Provides secure login and registration using	JWT, bcrypt

	encrypted passwords and token-based authentication.	
Admin Management	Allows administrators to manage users, bookings, and cab fleet from the admin dashboard.	React Admin Panel + Node API

System Data Flow

The UCab system follows a three-tier architecture consisting of the presentation layer (frontend), application layer (backend), and data layer (database). The interaction process is described below:

1. User opens the UCab website through a browser.

2. The frontend interface built with React.js displays the booking options.

3. When a user performs an action such as login or booking a cab, the request is sent to the backend server.

4. The backend server processes the request and validates the data.

5. The backend communicates with MongoDB database to store or retrieve data.

6. The processed result is sent back to the frontend and displayed to the user.

Solution Architecture Diagram (Logical Representation)

```
        User / Browser
              |
              v
        React Frontend
     (UI, Booking Pages)
              |
              v
       Express / Node API
   (Authentication, Booking Logic)
              |
              v
            MongoDB
       (Users, Cabs, Bookings)
```

This architecture ensures separation of concerns where the frontend handles user interaction, the backend processes business logic, and the database securely stores application data.